

Kopiering av strängar

Då man vill kopiera strängar skiljer sig C-strängar mycket från `string`. Kopieringen kan relativt enkelt gå fel om man inte är försiktig. Orsaken till detta är att man måste manuellt göra all minneshantering och gör amn ett fel där kan hela programmet krascha. Vi kan göra en enkel funktion som utför en kopiering:

```
#include <iostream>

void copy (char * Source, char * Destination) {
    // iterera över och kopiera alla tecken
    for ( unsigned int Index = 0; Index <= strlen ( Source ); Index++ ) {
        // kopiera ett tecken
        Destination[Index] = Source[Index];
    }
}

int main () {
    char Hello1[5] = "Hej!";
    char Hello2[5];

    // utför kopieringen
    copy ( Hello1, Hello2 );

    cout << "Hello1='" << Hello1 << "', Hello2= '" << Hello2 << "'" << endl;
}
```

Notera att vi i funktionen `copy()` kopierar ett tecken mera än vad strängen är lång. Detta görs för att få med den sista terminerande `\0`, annars är strängen oterminerad, vilket leder till provlem senare. Vi gör heller ingen som helst kontroll på att `Destination` kan rymma alla tecken som finns i `Source`. Detta är upp till anroparen att kontrollera. Vad händer om vi försöker kopiera in i en för kort sträng? Vi modifierar programmet ovan så att det utför en felaktig kopiering:

```
int main () {
    char Hello1[] = "Hej! Detta är en lååång sträng.";
    char Hello2[1];

    // utför kopieringen
    copy ( Hello1, Hello2 );

    cout << "Hello1 = '" << Hello1 << "', " << endl
         << "Hello2 = '" << Hello2 << "'" << endl;
}
```

Det använder samma funktion `copy()` som ovan. Resultatet av körningen blev på den använda maskinen:

```
% ./CStringCopy3
Hello1 = 'ej! Detta är en lååång sträng.',
Hello2 = 'Hej! Detta är en lååång sträng.'
%
```

Notera att `Hello1` har tappat det första H:et. Men vid olika körningar och med olika längder på

`Hello2` hände antingen ingenting eller så kraschade programmet med en `segmentation fault`. Detta illustrerar att det kan fungera helt fint att köra ett program som hanterar strängar fel, och felet kan visa sig på helt andra ställen än där det gjordes. Orsaken här är att `Hello2` delvis skriver över `Hello1`, d.v.s. att i minnet finns `Hello2` direkt före `Hello1` och endast det första tecknet (ett "H") hamnar in i `Hello2` och resten kommer i `Hello1` (börjande från "e"). I vårt fall var det lätt att hitta felet, men tänk om vi istället hade korrumperat någon annan helt orelaterad variabel eller objekt. Dylika fel kan vara mycket svåra att hitta.

En bättre version av ovanstående kan göras ifall vi manuellt allokerar så mycket minne som behövs för den nya strängen samt dess terminator. Vi kan i så fall skriva om `main()` från ovan så att det blir:

```
int main () {
    char Hello1[] = "Hej! Detta är en lååång sträng.";
    char * Hello2;

    // allokerar minne
    Hello2 = new char [ strlen ( Hello1 ) + 1 ];

    // utför kopieringen
    copy ( Hello1, Hello2 );

    cout << "Hello1 = '" << Hello1 << "', " << endl
          << "Hello2 = '" << Hello2 << "'" << endl;

    delete [] Hello2;
}
```

Enda skillnaden är att `Hello2` är en pekare och inte en vektor och att vi allokerar så mycket minne som behövs före kopieringen. Detta är det korrekta sättet att utföra en kopiering.

Färdiga funktioner

Det finns några färdiga funktioner som gör livet lättare då man vill kopiera C-strängar. Den ena och mycket använda funktionen är `strcpy()`, som kopierar en sträng till en annan. Den fungerar på samma sätt som vår funktion `copy()` med de enda skillnaderna att den även returnerar destinationssträngen och att parametrarna är omsvängda. `strcpy()` finns definierad i `<string.h>`. Även här måste man försäkra sig om att destinationssträngen innehåller tillräckligt med utrymme för hela den kopierade strängen, annars får man liknande fel som med `copy()`. Vi kan skriva om vårt kopierande program från ovan:

```
#include <string.h>

int main () {
    char Hello1[] = "Hej! Detta är en lååång sträng.";
    char * Hello2;

    // allokerar minne
    Hello2 = new char [ strlen ( Hello1 ) + 1 ];

    // utför kopieringen
    strcpy ( Hello2, Hello1 );

    cout << "Hello1 = '" << Hello1 << "', " << endl
          << "Hello2 = '" << Hello2 << "'" << endl;

    delete [] Hello2;
```

I övrigt är `strcpy()` enkel att använda. Det finns en alternativ version av `strcpy()` som heter `strncpy()` som kopierar ett visst antal tecken från en sträng. Där måste man dock vara noggrann med att man manuellt ser till att terminatören hamnar på rätt plats, eftersom `strncpy()` inte automatiskt placerar en dylik i destinationssträngen. Vi kan använda oss av `strncpy()` för att kopiera substrängar.

```
#include <iostream>
#include <string.h>

int main () {
    char Hello1[] = "Hej! Detta är en lååång sträng.";
    char * Hello2;

    // allokera minne
    Hello2 = new char [ strlen ( "Detta" ) + 1 ];

    // utför kopieringen
    strncpy ( Hello2, Hello1 + 5, 5 );
    Hello2[5] = '\0';

    cout << "Hello1 = '" << Hello1 << "', " << endl
          << "Hello2 = '" << Hello2 << "'" << endl;

    delete [] Hello2;
}
```

Vi vill här kopiera en substräng som börjar på det femte tecknet i `Hello1` och är fem tecken lång. Vi använder oss av normal pekararitmetik för att ange minnespositionen för det femte tecknet i `Hello1`. Den sista parametern är antalet tecken som skall kopieras. Vi måste här manuellt se till att `Hello2` termineras med `\0` eftersom vi inte kopierade med ett sådant tecken. Notera även att vi kan kontrollera längden på en strängkonstant med hjälp av `strlen()` för att undvika att vi räknar för hand och gör ett fel. Kör man ovanstående program får man denna utskrift:

```
% ./CStringSubstring1
Hello1 = 'Hej! Detta är en lååång sträng.',
Hello2 = 'Detta'
%
```

En alternativ funktion som kan användas är `strdup()` som duplicerar en sträng. Enda problemet här är att `strdup()` använder sig av `malloc()` för att göra en allokering för den nya strängen, och den allokerade strängen skall således frigöras med `free()`. Mera information om dessa på [avsnittet Minneshantering på C:s vis i Kapitel 12](#). Använder man `strdup()` får man alltså inte använda `delete` för att frigöra minnet. Ett exempel:

```
#include <iostream>
#include <string.h>
#include <stdlib.h>

int main () {
    char * S1 = "Hello world";
    char * S2;

    // utför kopieringen
    S2 = strdup ( S1 );

    cout << "S1 = '" << S1 << "', " << endl
          << "S2 = '" << S2 << "'" << endl;
```

```
// frigör strängen med C:s funktion free()
free ( S2 );
}
```

Man bör dock undvika `strdup()` för att inte blanda ihop allokeringar gjorda med `malloc()` och `new`.

[Förutgående](#)

Längden av strängar

[Hem](#)

[Upp](#)

[Nästa](#)

Konkatenering av strängar